A.T.O.M: Ankara Telecom Optimization Model

An Academic Engineering Report on Demand-Aware RF Propagation Simulation for Ankara

Author: Berk Unsal
Project: A.T.O.M, Ankara Telecom Optimization Model
Study Area: Ankara core bounding box, Turkey

Abstract

A.T.O.M is a full-stack geospatial planning system for modeling cellular signal propagation across Ankara, Turkey. The project combines OpenStreetMap building footprints, OpenCellID-derived planning records, deterministic radio propagation, demand-aware optimization, multi-cell interference analysis, scenario comparison, and field-measurement residual evaluation. The system evolved from a coverage-only model into a residential-density-aware planning product that reduces the tendency to optimize toward long empty corridors. This report documents the datasets, implementation, mathematical model, optimization process, and measured improvements produced by the demand surface.

1. Introduction

Urban cellular planning is difficult because radio propagation is not determined by distance alone. Buildings, dense housing clusters, commercial facilities, and public institutions all affect where network capacity is most valuable. This is especially important for 5G mmWave and future 6G Sub-THz systems, where high frequencies experience severe path loss and wall penetration penalties.

A.T.O.M focuses on Ankara because the city contains a useful mixture of dense residential districts, government and university campuses, commercial corridors, and lower-density peripheral land. The goal is not only to draw theoretical coverage rays, but to estimate where antenna sectors should point when serving meaningful demand.

2. Datasets

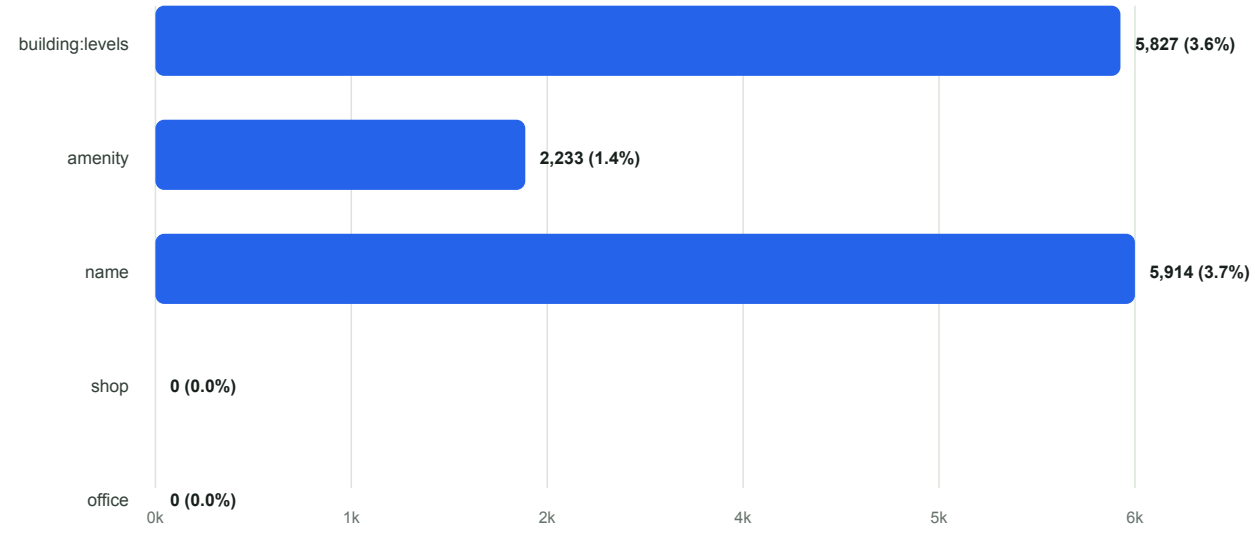
The system uses static local files at runtime. This avoids external API limits and makes simulations reproducible.

Dataset	Source	Runtime Format	Purpose
Cellular nodes	OpenCellID-derived extraction	GeoJSON	Tower/node placement for simulation origins
Buildings	OpenStreetMap via OSMnx	GeoJSON	Ray obstruction polygons and demand surface basis
Derived demand fields	Local Python enrichment	GeoJSON properties	POI, residential, and density-weighted optimization

2.1 Building Dataset Coverage

The current local export contains **161,626** building footprints. OSM metadata is useful but sparse: `building:levels` exists for 5,827 buildings, `amenity` for 2,233, and `name` for 5,914. The current local export has no retained `shop` or `office` values, so the final optimizer compensates with building-type, name, area, and density heuristics.

OSM Metadata Coverage in the Local Export



Source: generated from data-pipeline/ankara_buildings.geojson

2.2 Derived Demand Fields

The final exported building file adds the following fields:

Field	Meaning
`demand_weight`	Explicit POI/commercial/critical-service demand
`residential_demand`	Population-serving demand from homes and dense residential clusters
`density_score`	Local building-density score computed from a 150 m neighborhood
`nearby_buildings`	Count of nearby building centroids within 150 m
`nearby_residential_buildings`	Count of nearby residential-like buildings within 150 m
`weight_reason` / `residential_reason`	Human-readable explanation of why a building was weighted

---

3. System Architecture

A.T.O.M is implemented as a local-first, full-stack geospatial application.

Layer	Technology	Responsibility
Data pipeline	Python, GeoPandas, Shapely, OSMnx	Extract source geometry and compute demand fields
Dataset pack	Manifest, GeoJSON, SHA-256	Record provenance, bounds, CRS, identity, and validated runtime files
Simulation backend	Go, Gin, Tidwall R-tree	Bound requests, index geometry, run propagation, optimization, interference, recommendation, and measurement engines
Frontend	React, Vite, Leaflet, IndexedDB	Focused map workspace, local projects/scenarios, comparison, evidence inspection, and reports
Deployment	Docker multi-stage build	Build React and Go into one versioned production container

The backend validates the active dataset manifest and hashes, then loads tower and building data at startup. Building bounding boxes are inserted into an in-memory R-tree, so each short ray segment only checks nearby candidate polygons instead of scanning the entire city. Browser projects retain exact requests, model metadata, summaries, and recent map layers without adding a server-side database.

---

4. Propagation Model

The core propagation model uses Free-Space Path Loss, antenna gain, receiver sensitivity, and frequency-dependent wall penetration.

4.1 Free-Space Path Loss

For distance in meters and frequency in GHz:

```
FSPL(dB) = 20 log10(distance_m) + 20 log10(frequency_GHz) + 32.45
```

Received power is computed from EIRP:

```
EIRP_dBm = TxPower_dBm + AntennaGain_dBi
Rx_dBm = EIRP_dBm - FSPL(dB) - cumulative_wall_loss
```

The system uses:

Parameter	Value
Receiver sensitivity	-115 dBm
Antenna beamforming gain	+25 dBi
Segment step size	25 m

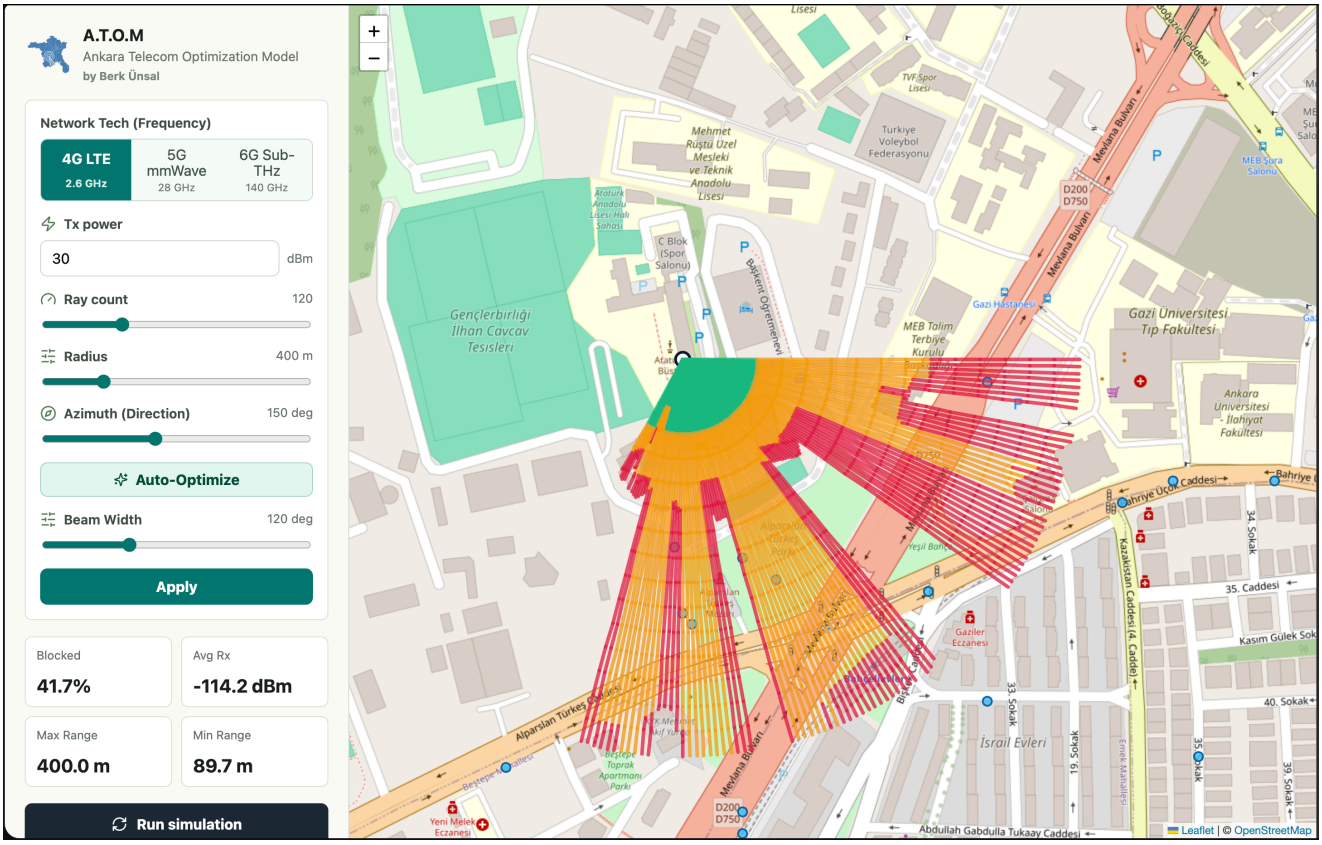
4.2 Frequency-Dependent Penetration

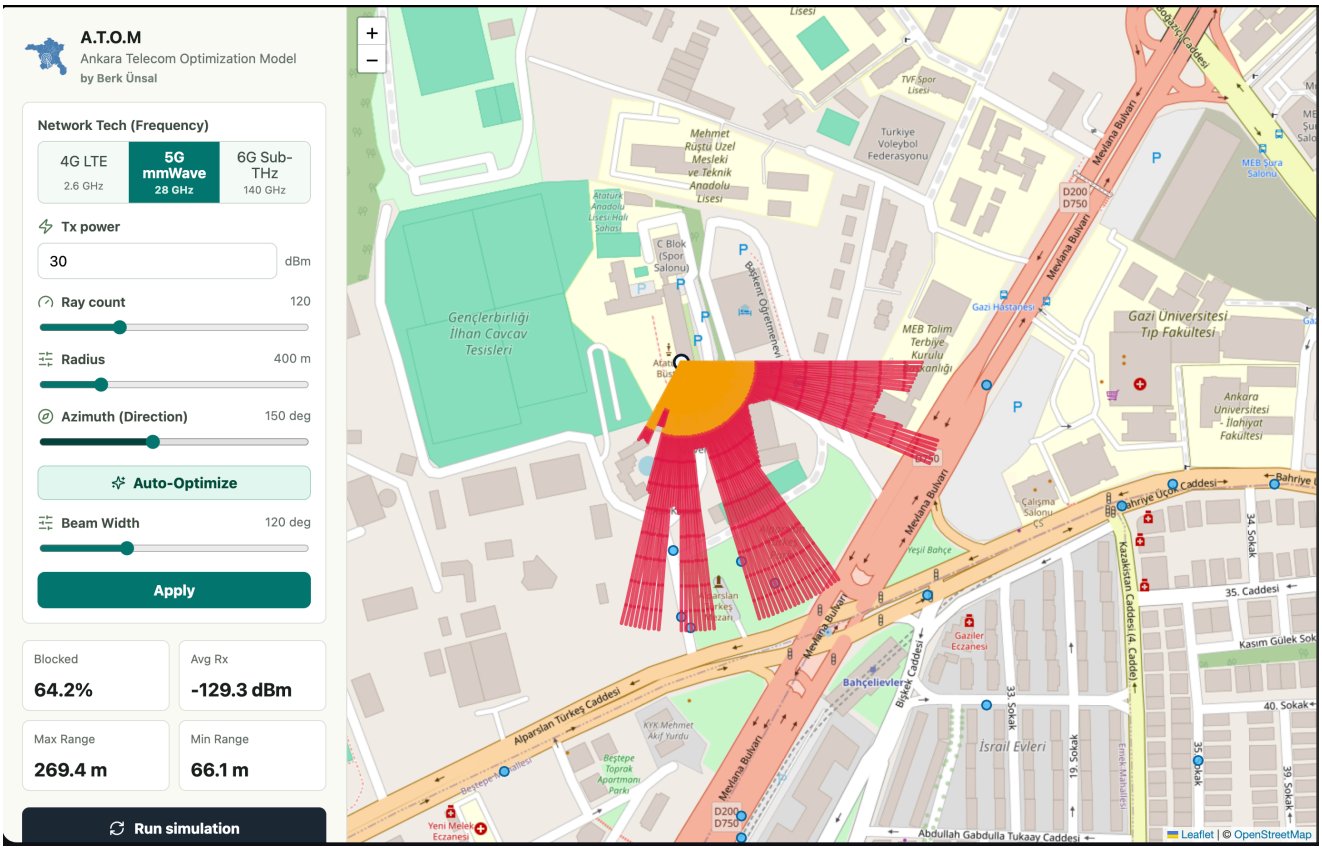
Wall attenuation depends on network generation:

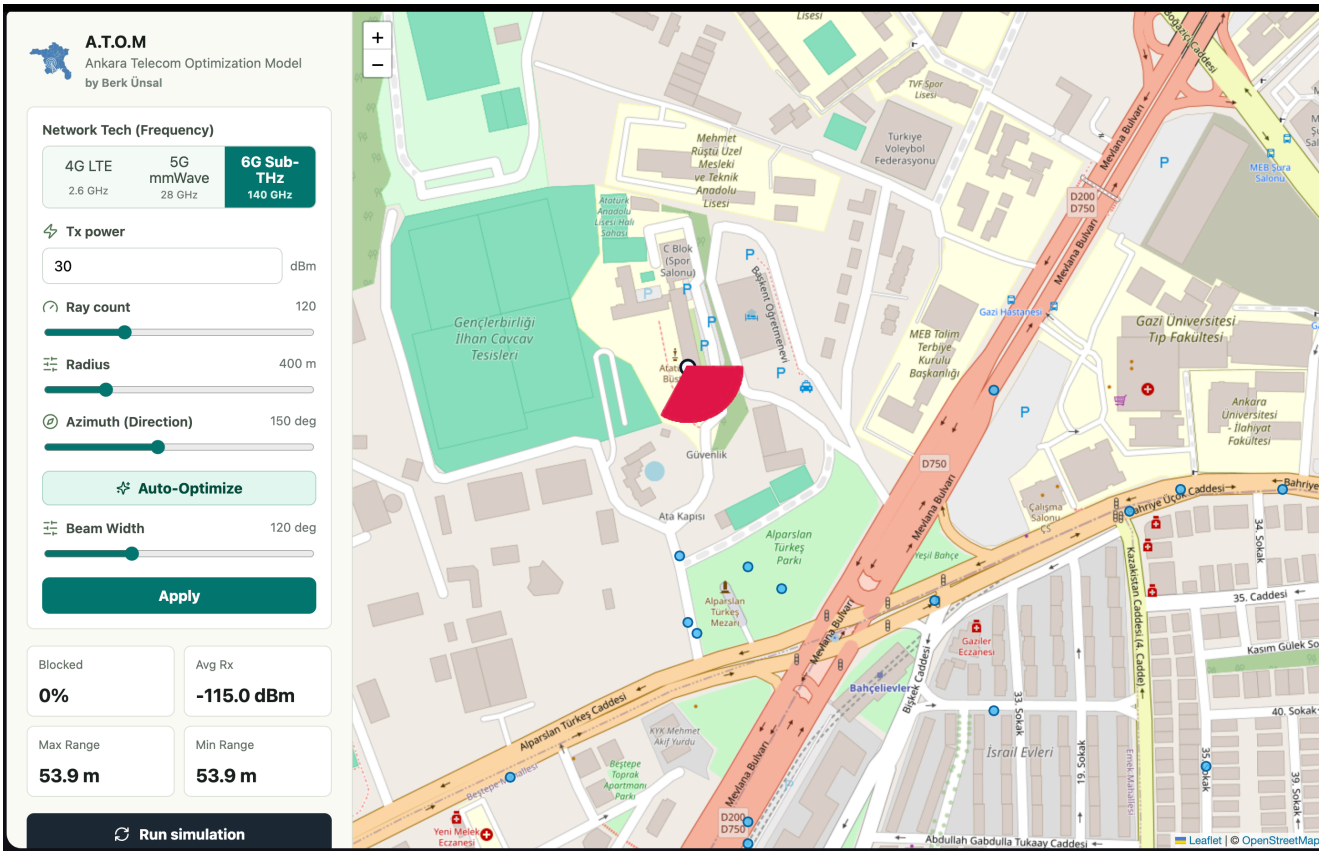
Technology	Frequency	Wall Loss Per Intersection	Expected Behavior
4G LTE	2.6 GHz	8 dB	Stronger penetration through buildings
5G mmWave	28 GHz	30 dB	Limited penetration, fast urban attenuation
6G Sub-THz	140 GHz	80 dB	Near-immediate blockage at walls

4.3 Segmented Beam Tracing

Each sector is divided into rays, and each ray is split into 25 m segments. Segments are colored by received power, which creates a visual heatmap rather than a single uniform line.


A.T.O.M
Ankara Telecom Optimization Model
by Berk Ünsal
Network Tech (Frequency)
4G LTE
2.6 GHz
5G mmWave
28 GHz
6G Sub-THz
140 GHz
Tx power
30
dBm
Ray count
120
Radius
400 m
Azimuth (Direction)
150 deg
Auto-Optimize
Beam Width
120 deg
Apply
Blocked
41.7%
Avg Rx
-114.2 dBm
Max Range
400.0 m
Min Range
89.7 m
Run simulation
Leaflet | © OpenStreetMap


A.T.O.M
Ankara Telecom Optimization Model
by Berk Ünsal
Network Tech (Frequency)
4G LTE
2.6 GHz
5G mmWave
28 GHz
6G Sub-THz
140 GHz
Tx power
30
dBm
Ray count
120
Radius
400 m
Azimuth (Direction)
150 deg
Auto-Optimize
Beam Width
120 deg
Apply
Blocked
64.2%
Avg Rx
-129.3 dBm
Max Range
269.4 m
Min Range
66.1 m
Run simulation
Leaflet | © OpenStreetMap



5. Optimization Methodology

The optimizer evolved through three major phases.

5.1 Phase 1: Geometry-Only Coverage

The initial optimizer maximized:

```
score = SUM(ray_length_m^2)
```

This made geometric sense, but it overvalued long unobstructed paths, including paths over low-demand or empty land.

5.2 Phase 2: POI and Commercial Demand

The second optimizer introduced `demand_weight` for buildings such as malls, hospitals, universities, schools, and commercial footprints:

```
score = coverage_score + demand_score
demand_score = SUM(unique_hit_building.demand_weight) * 10000
```

This improved prioritization of high-value targets but still undercounted homes because most residential buildings lacked rich OSM tags.

5.3 Phase 3: Residential-Density Surface

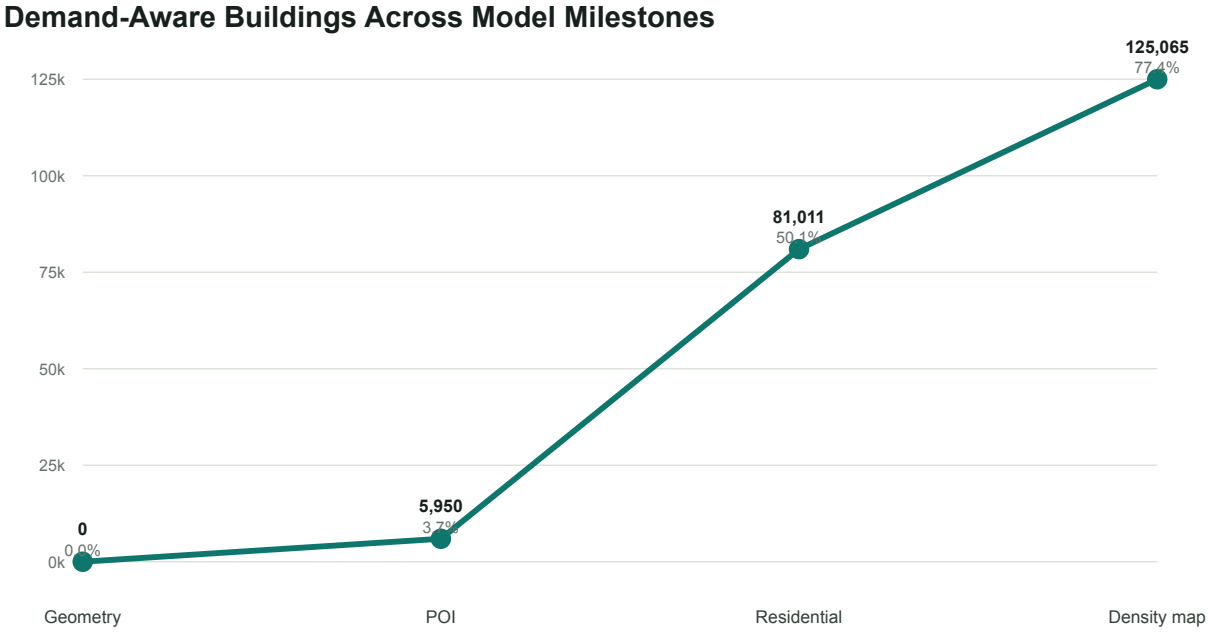
The final model adds a residential demand layer:

```
score = demand_score + residential_score + coverage_tiebreaker
```

The coverage component is now capped and normalized as a tie-breaker. This prevents a long clear ray over empty land from dominating a shorter ray serving dense housing.

Residential demand is assigned from:

- Residential building types such as `apartments`, `residential`, `house`, `detached`, and `semidetached_house`.
- Building levels when available.
- Local density within a 150 m neighborhood.
- Dense generic `building=yes` clusters that likely represent settlement patterns.

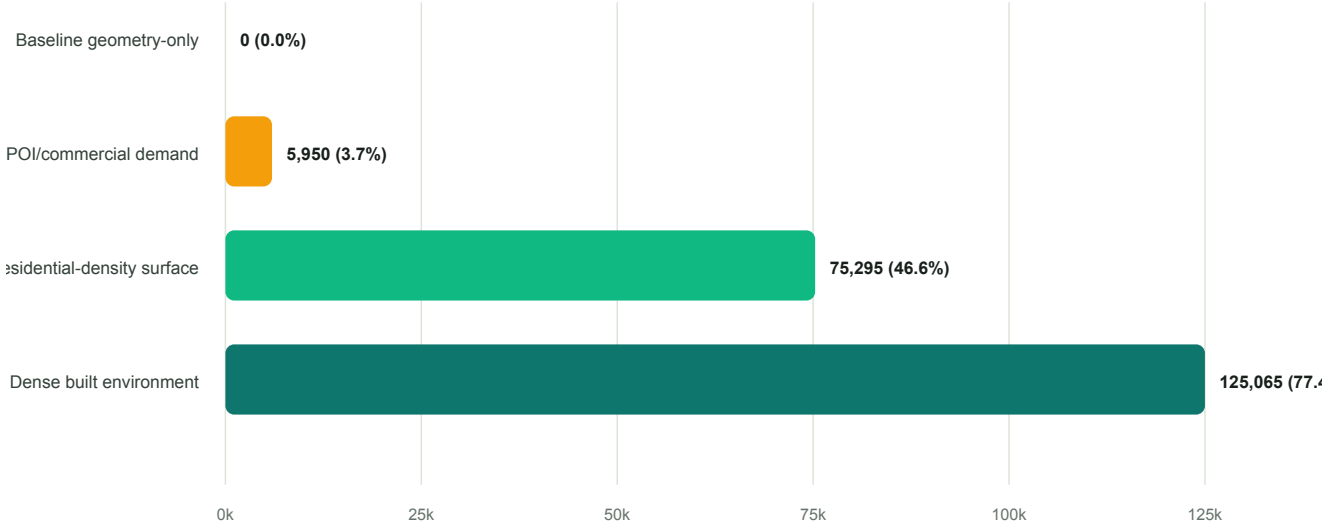


6. Results and Before/After Analysis

The most important improvement is the number of buildings that the optimizer can interpret as demand.

Phase	Demand-Aware Buildings	Share of Dataset	Interpretation
Baseline geometry-only	0	0.0%	Buildings block signal but do not represent demand
POI/commercial demand	5,950	3.7%	High-value facilities and explicit demand targets are recognized
Residential-density surface	75,295	46.6%	Homes and dense residential clusters influence optimization
Dense built environment	125,065	77.4%	Local settlement density is available as a planning feature

Demand Visibility Before and After Optimization

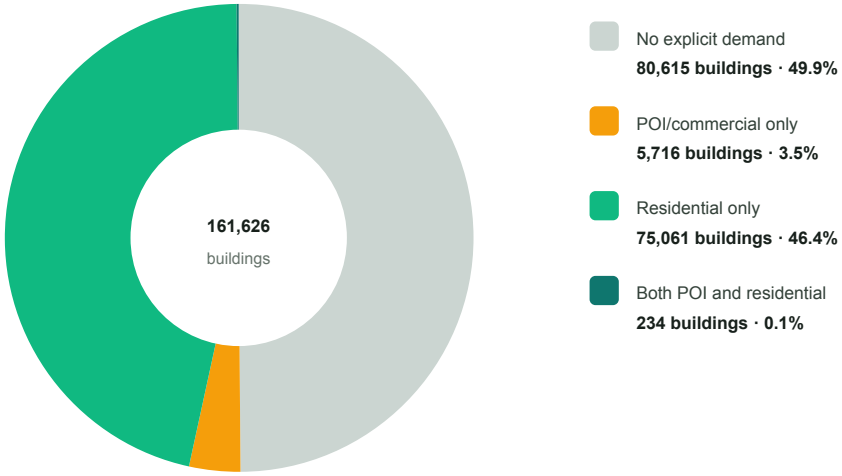


Source: generated from data-pipeline/ankara_buildings.geojson

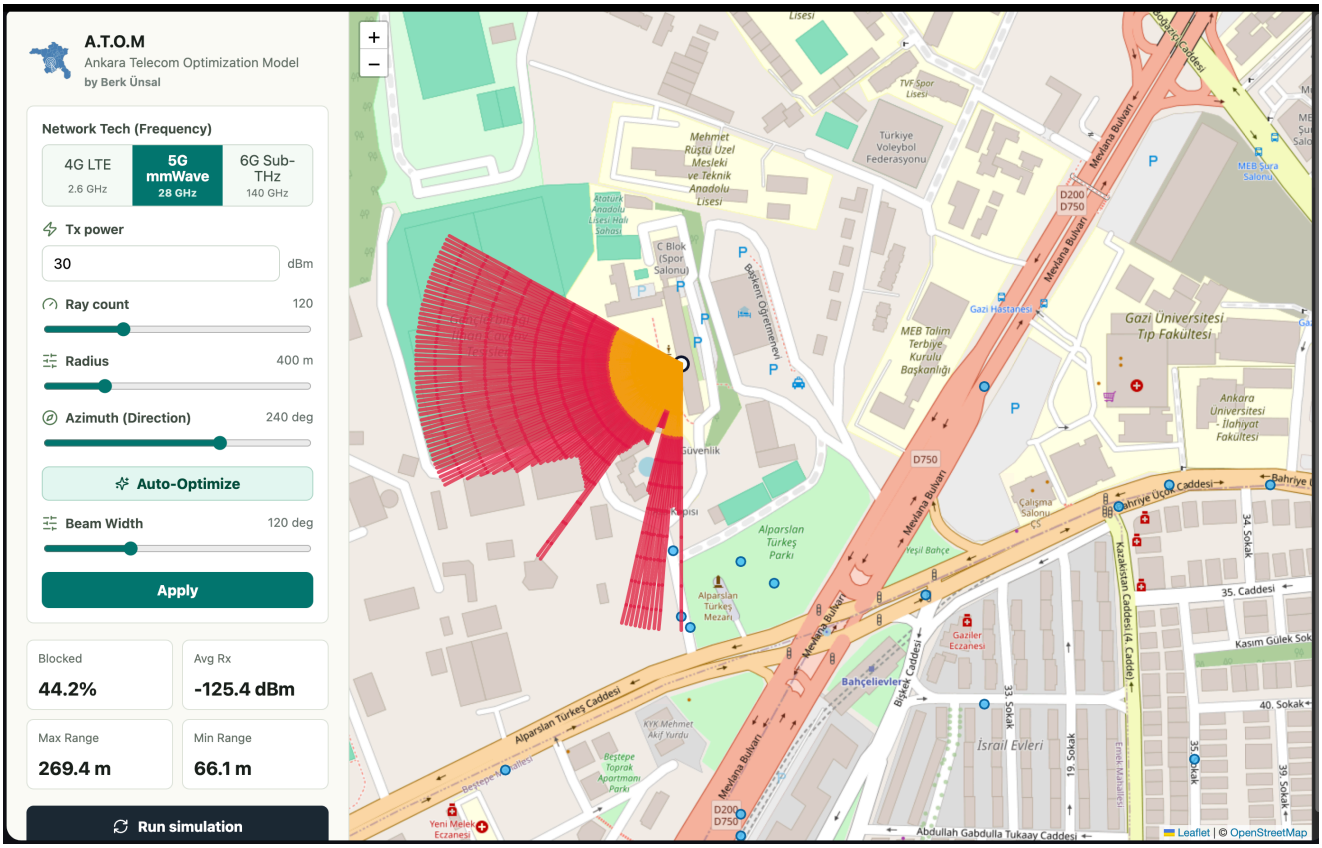
The final demand classification is:

Category	Count
No explicit demand	80,615
POI/commercial only	5,716
Residential only	75,061
Both POI and residential	234

Final Building Demand Classification



The result is a more realistic optimizer: high-value facilities still matter, but dense housing is no longer invisible. This makes the model better suited for population-serving antenna planning.



---

7. Implementation Notes

7.1 Python Data Pipeline

The Python pipeline performs:

1. Tower extraction and filtering from OpenCellID-derived data.
2. Building footprint export from OSMnx/OSM.
3. Offline enrichment of building footprints.
4. Demand and density scoring.
5. GeoJSON export for static runtime loading.

The report charts are generated from the same GeoJSON file using a dependency-free SVG generator:

```
python3 docs/report/generate_report_charts.py
```

7.2 Go Backend

The Go backend performs:

- GeoJSON parsing.
- R-tree spatial indexing.
- Concurrent ray simulation with goroutines.
- Segment-polygon intersection checks.
- Frequency-dependent penetration loss.
- Beamforming and azimuth optimization.

The optimizer returns:

```
{
  "optimal_azimuth": 240,
  "coverage_score": 1200,
  "demand_score": 450000,
  "residential_score": 900000,
  "hit_demand_buildings": 12,
  "hit_residential_buildings": 40,
  "data_quality": "good"
}
```

7.3 React Frontend

The frontend provides:

- A map-first command bar, workflow rail, focused tool drawer, and persistent Inspector.
- Single-cell and two-to-six-cell planning with explicit Run, Evaluate, Optimize, and Analyze actions.
- Independent GeoJSON layers for rays, gaps, interference, communication paths, candidates, and measurement residuals.
- Versioned local projects, named scenario snapshots, stale-result detection, import/export, and two-scenario comparison.
- Dataset/model provenance, measurement holdout evidence, calibration state, and report export.

---

8. Discussion and Limitations

The final model is stronger than the initial geometry-only optimizer, but it remains an approximation.

Limitation	Impact	Future Remedy
Sparse OSM metadata	Some shops/offices are missing from the current local export	Preserve richer OSM tags during fresh export
Synthetic residential demand	Density is an inferred proxy for population	Add GHSL, WorldPop, or municipality population grids
Simplified RF physics	No full diffraction, reflection, or multipath model	Add calibrated propagation models and field measurements
Static environment	No traffic, mobility, or temporal demand	Add authoritative time-dependent demand only when available
Global bias correction	One robust dB offset cannot identify separate wall, clutter, or terrain errors	Expand calibration only with sufficient field evidence
Candidate-site evidence	RF ranking does not establish ownership, permitting, cost, power, or backhaul	Add criteria only from authoritative dataset-pack inputs

The density surface is intentionally transparent: every weighted building records why it was assigned demand. This makes the model inspectable even when external population data is unavailable.

---

9. Conclusion

A.T.O.M demonstrates how a local geospatial simulation engine can combine RF propagation physics with demand-aware optimization. The project moved from pure ray coverage to a richer planning model that accounts for critical POIs, commercial sites, residential buildings, and local settlement density. The final model recognizes 75,295 residential-demand buildings and 5,950 POI/commercial-demand buildings, substantially improving the optimizer’s ability to point sectors toward places where service is valuable rather than merely where rays travel farthest.

The current product can compare predictions with uploaded 4G/5G RSRP samples and hold out evidence before applying a global bias correction. Future development should deepen that validation with redistributable measurement sets and replace synthetic residential proxies with authoritative population data. Even in its local-first form, A.T.O.M provides a strong foundation for explainable cellular network planning in dense urban environments.

Reproducibility

Regenerate charts:

```
python3 docs/report/generate_report_charts.py
```

Regenerate enriched building data offline:

```
data-pipeline/.venv/bin/python data-pipeline/fetch_buildings.py \
  --offline-input data-pipeline/ankara_buildings.geojson \
  --output data-pipeline/ankara_buildings.geojson
```

Run validation:

```
PYTHONPYCACHEPREFIX=/private/tmp/atom-pycache data-pipeline/.venv/bin/python -m
        py_compile \
  data-pipeline/export_ankara_buildings.py data-pipeline/fetch_buildings.py

cd backend-go
GOCACHE=/private/tmp/atom-gocache go test ./...
cd ..

cd frontend-react
npm run build
```